AMBIGUITY IN GRAMMAR

In a CFG that specifies the syntax of programming language or the rules for constructing an algebraic expression, interpreting a string correctly requires finding a correct derivation of the string in the grammar. A natural way of exhibiting the structure of derivation is to draw a derivation tree.

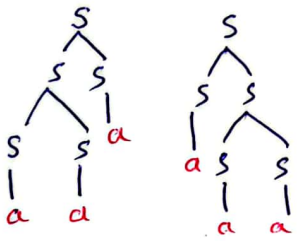
A CFG is ambiguous if there is at least one string in L(G) having two or more distinct derivation trees (or equivalently two or more distinct leftmost derivation).

Ambiguity is one undesirable property of CFG that we might wish to eliminate

- Consider the grammar

$$S \rightarrow SS \mid a$$

the word $w = aaa$, can be derived with two ways:

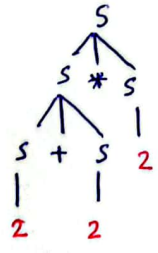


So, the grammar is ambiguous.

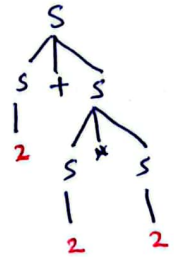
- Consider the CFG for algebraic expression as

$$S \rightarrow S+S \mid S*S \mid S*S \mid S/S \mid (S) \mid 2$$

the word $2+2*2$ can be derived with 2 ways:



Answer = 8



answer = 6 (Correct)

There is no general method (algorithm) to convert ambiguous grammar to unambiguous grammar. We have to find some strategy which may or may not be successful.

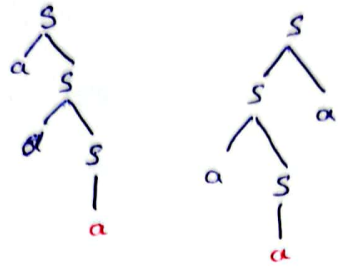
Common reasons:

1. Common prefix / suffix
2. Presence of right and left recursion simultaneously
3. and many more.

Example #1

$$S \to aS \mid Sa \mid a$$

ambiguous as, two derivation trees for $w = aaa$



unambiguous:

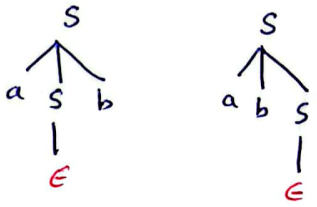
$$S \to aS \mid a$$

There is only one derivation for all words of a grammar, so grammar is unambiguous.

Example #2

$$S \to aSb \mid abS \mid \wedge$$

ambiguous as for $w = ab$



· Here ambiguity is only because of $\wedge$-production.

unambiguous form:

$$S \to aSb \mid abS \mid ab$$

as $\wedge$ is part of the language we can not eliminate it.

or

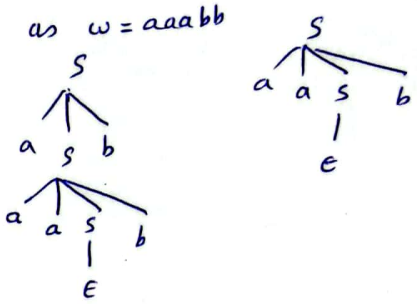
keeping $\wedge$-word in the language, we modified it as:

$$S \to T \mid \wedge$$
$$T \to aTb \mid abT \mid ab$$

Example #3

$$S \to aaSb \mid aSb \mid \wedge$$

as $w = aaabb$



unambiguous:

$$S \to aaSb \mid T$$
$$T \to aTb \mid \wedge$$

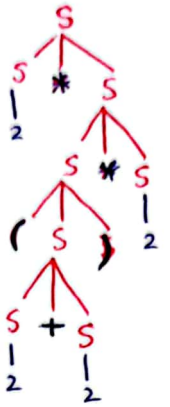
AMBIGUOUS AND UNAMBIGUOUS FORM OF ALGEBRAIC EXPRESSION

Example:

CFG for Algebraic Expression:

$$S \rightarrow S+S \mid S-S \mid S*S \mid S/S \mid (S) \mid a \mid 2$$

Derivation for $2*(2+2)*2$



$G = (V, \Sigma, S, P)$

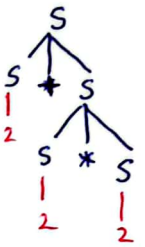
$V = \{S\}$

$\Sigma = \{a, 2, (,), +, -, *, /\}$

$S = S$ (start variable)

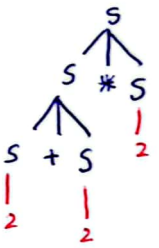
$P = \{ S \rightarrow S+S,\ S \rightarrow S-S,\ S \rightarrow S*S,\ S \rightarrow S/S,\ S \rightarrow (S),\ S \rightarrow a,\ S \rightarrow 2 \}$

This grammar is ambiguous as, there are two derivation trees for the expression $2+2*2$



Evaluates 6

DERIVATION 1



Evaluates 8 (which is wrong)

DERIVATION 2

UNAMBIGUOUS FORM

Here we need hierarchy of three levels:

Expression are:

Sums of terms

and Terms are:

Product of factors

Precedence Rules

lowest	1.	+, −
↓	2.	*, /
Highest	3.	( )

$S \rightarrow S+T \mid T$ → to bypass if expression like $2*2*2$

$T \rightarrow T*F \mid F$ → to bypass if expression like $(-)$ or 2 etc.

$F \rightarrow (S) \mid 2 \mid a$

T: Expression that Cannot itself be expressed as Sum (only for products)

F: Expression that cannot be expression as Sum of Product (used only for ( ), 2, a etc)

Associative Rule

+ , − Left to right

* , / Left to right

L to R Associativity

$S \rightarrow S + T$

Recursion (left recursive)

⟹

R to L Associativity

$S \rightarrow T + S$

right recursive

Example of right to Left associativity

2^{3^4} or $2 \uparrow 3 \uparrow 4$

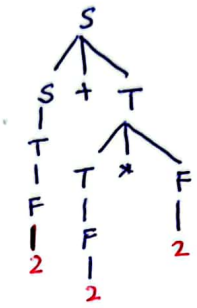
$S \rightarrow S+T \mid S-T \mid T$

$T \rightarrow T * F \mid T/F \mid F$

$F \rightarrow (S) \mid 2 \mid a$

This is unambiguous form, generates single derivation tree for all algebraic expression.

2 + 2 * 2



only one way to generate 2+2*2

Precedence Rules

Lowest 1. + , −

2. * , /

Highest 3. ( )

Note: Lowest precedence will be the first part in expression Highest precedence will be at low level in derivation tree.

Derivation Tree for 2 + a * (2 + 2 * 2) + a